

API v2 - Portal Academico CTC Salto

Base URL: `/v2` Swagger: `/docs` | Scalar: `/docs-scalar` Autenticacion: Bearer Token JWT (header `Authorization: Bearer <token>`)

Tabla de contenidos

1. Autenticacion
2. Portal Estudiante
3. Portal Docente
4. Admin - Programas
5. Admin - Materias
6. Admin - Instancias de Cursado
7. Admin - Instancias de Evaluacion
8. Admin - Instancias de Examen
9. Admin - Politicas de Calificacion
10. Admin - Politicas de Examen
11. Admin - Previaturas
12. Admin - Periodos de Inscripcion
13. Admin - Docentes por Materia
14. Admin - Inscripciones
15. Admin - Examenes
16. Admin - Documentos
17. Enums y valores posibles

1. Autenticacion

Prefijo: `/v2/auth`

POST `/v2/auth/google/login`

Inicia el flujo OAuth. Redirige a Google.

- **Auth:** No requiere
- **Response:** Redirect a Google

GET `/v2/auth/google/callback`

Callback de Google. Recibe el code, valida dominio @ctcsalto.edu.uy, obtiene OU, crea/actualiza usuario.

- **Auth:** No requiere
- **Response 200:**

```
{
  "access_token": "eyJhbGciOi...",
  "token_type": "bearer",
  "usuario": {
    "id": 1,
    "google_id": "google123",
```

```

    "moodle_id": 45,
    "email": "juan@ctcsalto.edu.uy",
    "nombre": "Juan",
    "apellido": "Perez",
    "foto_url": "https://lh3.google.com/foto.jpg",
    "ou_google": "/Alumnos",
    "rol": "estudiante",
    "activo": true,
    "google_activo": true,
    "moodle_activo": true,
    "fecha_creacion": "2026-03-01T10:00:00",
    "ultimo_acceso": "2026-05-04T15:30:00"
  }
}

```

POST /v2/auth/logout

Revoca el token JWT (blacklist en Redis).

- **Auth:** Bearer Token
- **Response 200:** { "message": "Sesion cerrada exitosamente" }

GET /v2/auth/me

Datos del usuario autenticado.

- **Auth:** Bearer Token
- **Response 200:** Objeto `UsuarioRead` (ver seccion callback arriba)

2. Portal Estudiante

Prefijo: /v2/portal/estudiante **Auth:** Bearer Token (rol `estudiante`)

GET /mi-perfil

Perfil del estudiante con datos de alumno.

- **Response 200:**

```

{
  "id": 1,
  "email": "juan@ctcsalto.edu.uy",
  "nombre": "Juan",
  "apellido": "Perez",
  "rol": "estudiante",
  "foto_url": "https://...",
  "activo": true,
  "perfil_alumno": {
    "alumno_id": 1,
    "fecha_ingreso": "2026-03-01"
  }
}

```

GET /mis-programas

Programas donde el alumno esta inscripto.

- Response 200:

```
[
  {
    "inscripcion_id": 1,
    "programa_id": 1,
    "nombre": "Tecnologo en Informatica",
    "tipo": "carrera",
    "area": "informatica",
    "estado": "activa",
    "anio_ingreso": 2026,
    "fecha_inscripcion": "2026-03-01T10:00:00"
  }
]
```

GET /programa/{programa_id}

Info de un programa con sus materias. Valida que el alumno este inscripto.

- Params: programa_id (path, int)
- Response 200: Programa con lista de materias (nombre, codigo, semestre, creditos)
- Error 403: "No estas inscripto en este programa"

GET /programa/{programa_id}/previaturas

Mapa de previaturas con estado del alumno en cada materia.

- Params: programa_id (path, int)
- Response 200:

```
[
  {
    "materia_id": 1,
    "nombre": "Programacion 1",
    "codigo": "P1",
    "semestre": 1,
    "estado_alumno": "aprobada",
    "previaturas": []
  },
  {
    "materia_id": 2,
    "nombre": "Programacion 2",
    "codigo": "P2",
    "semestre": 2,
    "estado_alumno": "cursando",
    "previaturas": [
      { "materia_previa_id": 1, "nombre": "Programacion 1", "tipo_requerido": "aprobada" }
    ]
  }
]
```

GET /mis-materias?anio_lectivo={anio}

Materias inscriptas en un ano lectivo.

- Query params: `anio_lectivo` (int, requerido)
- Response 200:

```
[
  {
    "inscripcion_id": 1,
    "materia_id": 1,
    "nombre": "Programacion 1",
    "codigo": "P1",
    "semestre": 1,
    "estado": "cursando",
    "nota_curso": null,
    "faltas": 2,
    "instancia_cursado_id": 1
  }
]
```

GET `/mi-materia/{inscripcion_id}`

Detalle completo de una materia inscripta (notas, faltas, calificaciones).

- Params: `inscripcion_id` (path, int)
- Response 200:

```
{
  "inscripcion_id": 1,
  "materia": { "nombre": "Programacion 1", "codigo": "P1", "semestre": 1, "creditos": 10 },
  "estado": "cursando",
  "nota_curso": 75.5,
  "nota_final": null,
  "faltas": 2,
  "faltas_maximas": 10,
  "calificaciones": [
    {
      "instancia_evaluacion_id": 1,
      "nombre": "Parcial 1",
      "peso_maximo": 50,
      "nota": 38.5,
      "fecha": "2026-04-15T10:00:00"
    }
  ]
}
```

GET `/mi-escolaridad?programa_id={id}`

Escolaridad completa del alumno en un programa.

- Query params: `programa_id` (int, requerido)
- Response 200:

```
[
  {
    "id": 1,
    "materia_nombre": "Programacion 1",
    "materia_codigo": "P1",
    "semestre": 1,
    "anio_lectivo": 2026,
    "estado": "exonerado",
    "nota_curso": 92.0,
    "nota_final": 92.0,
  }
]
```

```
    "creditos_obtenidos": 10,  
    "faltas": 1  
  }  
]
```

GET /materias-disponibles?programa_id={id}&anio_lectivo={anio}

Materias a las que el alumno puede inscribirse (valida previaturas cumplidas).

- Query params: `programa_id` (int), `anio_lectivo` (int) - ambos requeridos
- Response 200:

```
[  
  {  
    "materia_id": 2,  
    "nombre": "Programacion 2",  
    "codigo": "P2",  
    "semestre": 2,  
    "instancia_cursado_id": 5,  
    "inscribible": true,  
    "motivo": null  
  },  
  {  
    "materia_id": 3,  
    "nombre": "Programacion 3",  
    "codigo": "P3",  
    "semestre": 3,  
    "instancia_cursado_id": null,  
    "inscribible": false,  
    "motivo": "Previaturas no cumplidas: Programacion 2"  
  }  
]
```

POST /inscribirse-materia

Inscribirse a una materia. Valida previaturas y periodo activo.

- Body:

```
{ "instancia_cursado_id": 5 }
```

- Response 200: Objeto `InscripcionMateriaRead`

```
{  
  "id": 1,  
  "usuario_id": 1,  
  "instancia_cursado_id": 5,  
  "estado": "cursando",  
  "nota_curso": null,  
  "nota_final": null,  
  "faltas": 0,  
  "creditos_obtenidos": 0,  
  "fecha_inscripcion": "2026-03-15T10:00:00"  
}
```

- Error 400: "Previaturas no cumplidas: ..." / "No hay periodo de inscripcion activo"

GET /mis-calificaciones/{inscripcion_id}

Calificaciones del alumno en una inscripcion.

- **Params:** `inscripcion_id` (path, int)
- **Response 200:**

```
[
  {
    "id": 1,
    "inscripcion_id": 1,
    "instancia_evaluacion_id": 1,
    "nota": 85.0,
    "docente_id": 3,
    "fecha": "2026-04-15T10:00:00",
    "observaciones": null
  }
]
```

POST /inscribirse-examen

Inscribirse a un examen. Valida estado A_EXAMEN y periodo de inscripcion abierto.

- **Body:**

```
{
  "inscripcion_materia_id": 1,
  "instancia_examen_id": 3
}
```

- **Response 201:** Objeto `InscripcionExamenRead`

```
{
  "id": 1,
  "inscripcion_materia_id": 1,
  "instancia_examen_id": 3,
  "fecha_inscripcion": "2026-07-01T10:00:00",
  "nota_examen": null,
  "estado": "inscripto"
}
```

GET /mis-examenes/{inscripcion_id}

Historial de examenes de una inscripcion a materia.

- **Params:** `inscripcion_id` (path, int)
- **Response 200:** Lista de `InscripcionExamenRead`

DELETE /desinscribir-examen/{inscripcion_examen_id}

Desinscribirse de un examen (solo si esta INSCRIPTO).

- **Response 204:** Sin body

DELETE /desinscribir-materia/{inscripcion_id}

Desinscribirse de una materia (solo si CURSANDO y dentro de periodo activo).

- Response 204: Sin body

GET /exámenes-disponibles?programa_id={id}

Exámenes disponibles para el alumno (materias en A_EXAMEN con inscripción abierta).

- Query params: `programa_id` (int, requerido)
- Response 200:

```
[
  {
    "instancia_examen_id": 3,
    "inscripcion_materia_id": 1,
    "materia_id": 1,
    "materia_nombre": "Programacion 1",
    "materia_codigo": "P1",
    "nombre_examen": "Febrero 2026",
    "fecha_examen": "2026-02-15",
    "hora": "09:00",
    "salon": "Salon A",
    "modalidad": "presencial",
    "tipo": "ordinario",
    "ya_inscripto": false
  }
]
```

GET /todos-mis-exámenes

Todos los exámenes del alumno, todas las materias, ordenados por fecha.

- Response 200:

```
[
  {
    "inscripcion_examen_id": 1,
    "inscripcion_materia_id": 1,
    "materia_nombre": "Programacion 1",
    "materia_codigo": "P1",
    "nombre_examen": "Febrero 2026",
    "fecha_examen": "2026-02-15",
    "estado": "aprobado",
    "nota_examen": 78.0
  }
]
```

GET /mi-egreso?programa_id={id}

Verificar progreso de egreso en un programa.

- Query params: `programa_id` (int, requerido)
 - Response 200: Estado de egreso con materias pendientes/aprobadas
-

3. Portal Docente

Prefijo: `/v2/portal/docente` Auth: Bearer Token (rol `docente` o `administrativo`)

GET `/mi-perfil`

Perfil del docente con datos de profesor.

- Response 200:

```
{
  "id": 3,
  "email": "profe@ctcsalto.edu.uy",
  "nombre": "Maria",
  "apellido": "Lopez",
  "rol": "docente",
  "perfil_profesor": {
    "profesor_id": 1,
    "cargo": "titular",
    "dedicacion": "tiempo_completo",
    "especialidad": "Programacion"
  }
}
```

GET `/mis-programas`

Programas donde el docente es coordinador.

- Response 200:

```
[
  {
    "programa_id": 1,
    "nombre": "Tecnologo en Informatica",
    "tipo": "carrera",
    "area": "informatica",
    "cantidad_materias": 25
  }
]
```

GET `/mis-materias?anio_lectivo={anio}`

Materias asignadas al docente en un año lectivo.

- Query params: `anio_lectivo` (int, requerido)
- Response 200:

```
[
  {
    "asignacion_id": 1,
    "instancia_cursado_id": 5,
    "materia_id": 1,
    "nombre": "Programacion 1",
    "codigo": "P1",
    "semestre": 1,
    "anio_lectivo": 2026,
    "salon": "Salon A",
  }
]
```



```
    "horario": "Lunes 18-21",
    "estado": "en_curso",
    "rol_docente": "titular"
  }
]
```

GET /instancia-cursado/{id}/alumnos

Lista de alumnos inscriptos en una instancia.

- **Params:** `instancia_cursado_id` (path, int)
- **Response 200:**

```
[
  {
    "inscripcion_id": 1,
    "usuario_id": 1,
    "nombre": "Juan",
    "apellido": "Perez",
    "estado": "cursando",
    "nota_curso": null
  }
]
```

GET /instancia-cursado/{id}/detalle

Detalle completo: info materia, alumnos, evaluaciones, faltas maximas.

- **Response 200:** Objeto con toda la info de la instancia

GET /instancia-cursado/{id}/resultados

Tabla de resultados (acta). Todos los alumnos con notas por evaluacion.

- **Response 200:** Tabla con alumnos, notas, estados

GET /instancia-cursado/{id}/estadisticas

Estadísticas de aprobacion y asistencia.

- **Response 200:**

```
{
  "total_inscriptos": 30,
  "aprobados": 15,
  "reprobados": 5,
  "exonerados": 8,
  "a_examen": 7,
  "abandonos": 2,
  "perdidos_inasistencia": 1,
  "promedio_notas": 68.5,
  "tasa_aprobacion": 76.7
}
```

GET `/instancia-cursado/{id}/calificaciones?instancia_evaluacion_id={id}`

Notas de todos los alumnos en una evaluacion especifica.

- **Query params:** `instancia_evaluacion_id` (int, requerido)
- **Response 200:** Lista con nota por alumno

POST `/instancia-cursado/{id}/calificaciones`

Cargar nota individual.

- **Body:**

```
{
  "inscripcion_id": 1,
  "instancia_evaluacion_id": 1,
  "nota": 85.5,
  "equipo_id": null,
  "observaciones": "Buen trabajo"
}
```

- **Response 200:** Objeto `CalificacionRead`

POST `/instancia-cursado/{id}/calificaciones/batch`

Carga masiva de notas para una evaluacion.

- **Body:**

```
{
  "instancia_evaluacion_id": 1,
  "calificaciones": [
    { "inscripcion_id": 1, "nota": 85.0 },
    { "inscripcion_id": 2, "nota": 72.0 },
    { "inscripcion_id": 3, "nota": 90.0, "observaciones": "Excelente" }
  ]
}
```

- **Response 200:**

```
{
  "exitosos": 3,
  "errores": [],
  "calificaciones": [ ... ]
}
```

POST `/instancia-cursado/{id}/nota-final-directa`

Cargar nota final sin parciales (promedio ya calculado).

- **Body:**

```
{
  "inscripcion_id": 1,
```

```
"nota": 78.0
}
```

- **Response 200:** Objeto `InscripcionMateriaRead` (estado recalculado automaticamente)

GET `/instancia-cursado/{id}/equipos?instancia_evaluacion_id={id}`

Equipos de una evaluacion grupal.

- **Query params:** `instancia_evaluacion_id` (int, requerido)
- **Response 200:** Lista de equipos con miembros

POST `/instancia-cursado/{id}/equipos`

Crear equipo para evaluacion grupal.

- **Body:**

```
{
  "instancia_evaluacion_id": 1,
  "nombre": "Equipo A",
  "miembros_ids": [1, 2, 3]
}
```

- **Response 200:** { "id": 1, "nombre": "Equipo A", "instancia_evaluacion_id": 1 }

DELETE `/instancia-cursado/{id}/equipos/{equipo_id}`

Eliminar equipo.

- **Response 204:** Sin body

POST `/instancia-cursado/{id}/faltas`

Registrar una falta a un alumno. Si alcanza `faltas_maximas`, cambia estado a `perdido_inasistencia` automaticamente.

- **Body:**

```
{ "inscripcion_id": 1 }
```

- **Response 200:** Objeto `InscripcionMateriaRead` (con faltas actualizado)

DELETE `/instancia-cursado/{id}/faltas`

Quitar una falta a un alumno (minimo 0).

- **Body:**

```
{ "inscripcion_id": 1 }
```

- **Response 200:** Objeto `InscripcionMateriaRead`

GET /materia/{materia_id}/examenes?instancia_examen_id={id}

Lista de inscriptos a examen para una instancia de examen.

- Query params: `instancia_examen_id` (int, requerido)
- Response 200: Lista con alumnos inscriptos, notas, estados

POST /materia/{materia_id}/examenes/{inscripcion_examen_id}/calificar

Calificar un examen.

- Body:

```
{ "nota_examen": 78.5 }
```

- Response 200: Objeto `InscripcionExamenRead` (estado actualizado)

POST /materia/{materia_id}/examenes/{inscripcion_examen_id}/ausente

Marcar como ausente en un examen.

- Response 200: Objeto `InscripcionExamenRead` con `estado: "ausente"`

4. Admin - Programas

Prefijo: `/v2/admin/programas` Auth: Bearer Token (rol `administrativo`)

POST /

Crear programa. Body:

```
{
  "nombre": "Tecnologo en Informatica",
  "tipo": "carrera",
  "area": "informatica",
  "duracion_semestres": 6,
  "total_creditos": 240,
  "descripcion": "Programa de 3 anos...",
  "coordinador_id": 1
}
```

Response 201: `ProgramaRead`

GET `/{{programa_id}}` - Obtener programa por ID

GET `/{{programa_id}}/con-materias` - Programa con lista de materias por semestre

GET `/` - Listar con paginacion (`?offset=0&limit=10`)

POST `/filters` - Filtros avanzados (body `Filter`)

PUT `/{{programa_id}}` - Actualizar (body parcial)

DELETE `/{{programa_id}}` - Eliminar (204)

5. Admin - Materias

Prefijo: `/v2/admin/materias` Auth: Bearer Token (rol `administrativo`)

POST `/`

Crear materia. Body:

```
{
  "programa_id": 1,
  "nombre": "Programacion 1",
  "codigo": "P1",
  "semestre": 1,
  "creditos": 10,
  "descripcion": "Introduccion a la programacion",
  "horas_semanales": 4,
  "activa": true
}
```

Response 201: `MateriaRead`

GET `/por-programa/{{programa_id}}` - Materias de un programa (ordenadas por semestre)

GET `/{{materia_id}}` - Obtener por ID

GET `/` - Listar con paginacion

POST `/filters` - Filtros avanzados

PUT `/{{materia_id}}` - Actualizar

DELETE `/{{materia_id}}` - Eliminar (204)

6. Admin - Instancias de Cursado

Prefijo: `/v2/admin/instancias-cursado` Auth: Bearer Token (rol `administrativo`)

POST `/`

Crear instancia de cursado. **Body:**

```
{
  "materia_id": 1,
  "anio_lectivo": 2026,
  "fecha_inicio": "2026-03-01T00:00:00",
  "fecha_fin": "2026-07-15T00:00:00",
  "salon": "Salon A",
  "horario": "Lunes 18-21",
  "cupos_maximo": 35,
  "faltas_maximas": 10,
  "estado": "planificada"
}
```

Response 200: `InstanciaCursadoRead`

GET `/?materia_id={id}&anio_lectivo={anio}` - Listar (filtros opcionales)

GET `/ {instancia_id}` - Obtener por ID

PUT `/ {instancia_id}` - Actualizar

DELETE `/ {instancia_id}` - Eliminar

7. Admin - Instancias de Evaluacion

Prefijo: `/v2/admin/instancias-evaluacion` Auth: Bearer Token (rol `administrativo`)

POST `/`

Crear instancia de evaluacion. **Body:**

```
{
  "instancia_cursado_id": 5,
  "nombre": "Parcial 1",
  "peso_maximo": 50,
  "orden": 1,
  "es_grupal": false,
  "fecha": "2026-04-15T10:00:00",
  "descripcion": "Primer parcial"
}
```

Response 201: `InstanciaEvaluacionRead`

GET `/instancia-cursado/{instancia_cursado_id}` - Evaluaciones de una instancia de cursado

POST `/filters` - Filtros avanzados

PUT `/instancia_id` - Actualizar

DELETE `/instancia_id` - Eliminar (204)

8. Admin - Instancias de Examen

Prefijo: `/v2/admin/instancias-examen` Auth: Bearer Token (rol `administrativo`)

POST `/`

Crear instancia de examen. Body:

```
{
  "materia_id": 1,
  "nombre": "Febrero 2026 - Programacion 1",
  "fecha_inicio_inscripcion": "2026-01-15T00:00:00",
  "fecha_fin_inscripcion": "2026-02-10T23:59:59",
  "fecha_examen": "2026-02-15T09:00:00",
  "hora": "09:00",
  "salon": "Salon A",
  "modalidad": "presencial",
  "tipo": "ordinario",
  "habilitado": true
}
```

Response 201: `InstanciaExamenRead`

GET `/materia/{materia_id}` - Instancias de examen de una materia

GET `/activas` - Instancias con inscripcion abierta

GET `/instancia_id` - Obtener por ID

PUT `/instancia_id` - Actualizar

DELETE `/instancia_id` - Eliminar

POST `/instancia_id/profesores` - Asignar profesor ({ `"docente_id": 3` })

GET `/instancia_id/inscriptos` - Lista de inscriptos al examen

9. Admin - Politicas de Calificacion

Prefijo: `/v2/admin/politicas-calificacion` Auth: Bearer Token (rol `administrativo`)

POST `/`

Crear politica. Body:

```
{
  "nombre": "Estandar 100 puntos",
  "escala_max": 100,
  "umbral_exoneracion": 86,
  "umbral_examen": 25,
  "umbral_aprobacion": 60,
  "cantidad_instancias": 2,
  "tipo_nota": "numerica",
  "descripcion": "Politica estandar del CTC"
}
```

Response 201: `PoliticaCalificacionRead`

GET `/politica_id` | GET `/` | POST `/filters` | PUT `/id` | DELETE `/id`

Logica de estados automaticos:

- `nota_curso >= 86` → `exonerado`
- `nota_curso >= 25 y < 86` → `a_examen`
- `nota_curso < 25` → `reprobado`

10. Admin - Politicas de Examen

Prefijo: `/v2/admin/politicas-examen` Auth: Bearer Token (rol `administrativo`)

POST `/`

Crear politica de examen. Body:

```
{
  "nombre": "Estandar examen",
  "escala_max": 100,
  "nota_aprobacion_examen": 60,
  "cantidad_oportunidades": 3,
  "descripcion": "3 oportunidades, aprueba con 60"
}
```

Response 201: `PoliticaExamenRead`

GET `/politica_id` | GET `/` | POST `/filters` | PUT `/id` | DELETE `/id`

11. Admin - Previaturas

Prefijo: `/v2/admin/previaturas` Auth: Bearer Token (rol `administrativo`)

POST `/`

Crear previatura. Valida ciclos y que ambas materias pertenezcan al mismo programa.

- Body:

```
{
  "materia_id": 2,
  "materia_previa_id": 1,
  "tipo_requerido": "aprobada"
}
```

- Response 201: `PreviaturaRead`
- Error 400: `"Se detectó un ciclo de previaturas"` / `"Las materias deben pertenecer al mismo programa"`

GET `/materia/{materia_id}`

Previaturas de una materia con nombres.

- Response 200:

```
[
  {
    "id": 1,
    "materia_id": 2,
    "materia_nombre": "Programacion 2",
    "materia_previa_id": 1,
    "materia_previa_nombre": "Programacion 1",
    "tipo_requerido": "aprobada"
  }
]
```

GET `/malla/{programa_id}`

Malla curricular completa agrupada por semestre con previaturas.

DELETE `/previatura_id` - Eliminar (204)

12. Admin - Periodos de Inscripcion

Prefijo: `/v2/admin/periodos-inscripcion` Auth: Bearer Token (rol `administrativo`)

POST `/`

Crear periodo de inscripcion. Body:

```
{
  "nombre": "Inscripciones Marzo 2026",
  "fecha_inicio": "2026-02-15T00:00:00",
  "fecha_fin": "2026-03-15T23:59:59",
  "anio_lectivo": 2026,
  "habilitado": true
}
```

Response 201: `PeriodoInscripcionMateriaRead`

GET `/_{id}` | GET `/` | POST `/filters` | PUT `/_{id}` | DELETE `/_{id}`

13. Admin - Docentes por Materia

Prefijo: `/v2/admin/docentes-materia` Auth: Bearer Token (rol `administrativo`)

POST `/`

Asignar docente a instancia de cursado. Body:

```
{
  "docente_id": 3,
  "instancia_cursado_id": 5,
  "rol_docente": "titular"
}
```

Response 201: `DocenteMateriaRead`

GET `/instancia-cursado/{instancia_cursado_id}` - Docentes asignados

PUT `/_{asignacion_id}` - Cambiar rol

DELETE `/_{asignacion_id}` - Desasignar (204)

14. Admin - Inscripciones

Prefijo: `/v2/admin/inscripciones` Auth: Bearer Token (rol `docente` o `administrativo`)

POST `/inscribir`

Inscripcion manual (admin, salta validacion de periodo). Body:

```
{
  "usuario_id": 1,
  "instancia_cursado_id": 5
}
```

Response 200: IncripcionMateriaRead

POST /marcar-inasistencia

Marcar alumno como perdido por inasistencia. Body:

```
{ "inscripcion_id": 1, "motivo": "Supero las faltas maximas" }
```

POST /marcar-abandono

Marcar alumno como abandono. Body:

```
{ "inscripcion_id": 1, "motivo": "No se presento mas" }
```

GET /escolaridad/{usuario_id}?programa_id={id}

Consultar escolaridad de cualquier alumno.

GET /verificar-egreso/{usuario_id}?programa_id={id}

Verificar si un alumno cumple requisitos de egreso.

15. Admin - Exámenes

Prefijo: /v2/admin/examenes Auth: Bearer Token (rol administrativo)

POST /inscribir

Inscribir estudiante a examen (sin validar periodo). Body:

```
{
  "inscripcion_materia_id": 1,
  "instancia_examen_id": 3
}
```

Response 201: IncripcionExamenRead

POST /{inscripcion_examen_id}/calificar

Calificar examen. Body:

```
{ "nota_examen": 78.5 }
```

Response 200: IncripcionExamenRead - Si aprueba, la inscripcion materia pasa a aprobado

POST `/incripcion_examen_id/ausente`

Marcar como ausente. La materia queda en `a_examen`.

GET `/instancia/{instancia_examen_id}` - Lista de inscripciones a examen

DELETE `/incripcion_examen_id` - Desinscribir (solo estado `inscripto`) (204)

16. Admin - Documentos

Prefijo: `/v2/admin/documentos` Auth: Bearer Token (rol `administrativo`)

POST `/usuario_id`

Subir documento para cualquier usuario. **multipart/form-data**:

- `archivo` (file, requerido): Archivo PDF o imagen
- `tipo` (string, requerido): Tipo de documento (`formula_69a` , `escolaridad` , `constancia_convenio` , `cedula` , `titulo` , `otro`)
- `descripcion` (string, opcional): Descripcion del documento

Response 201:

```
{
  "id": 1,
  "usuario_id": 5,
  "tipo": "escolaridad",
  "nombre_original": "escolaridad.pdf",
  "mime_type": "application/pdf",
  "tamano_bytes": 45000,
  "descripcion": "Escolaridad 2026",
  "subido_por": 1,
  "fecha_subida": "2026-05-17T10:00:00",
  "activo": true,
  "id_rastreo": "uuid..."
}
```

GET `/usuario_id`

Listar documentos de un usuario. Query params:

- `tipo` (opcional): Filtrar por tipo de documento

Response 200: `DocumentoUsuarioRead[]`

GET `/descargar/{documento_id}`

Descargar cualquier documento. Response 200: Archivo binario

DELETE `/documento_id`

Eliminar documento (soft delete). Response 204

Endpoints de documentos en Portal Estudiante

Prefijo: /v2/portal/estudiante

Metodo	Ruta	Descripcion
POST	/documentos	Subir documento propio (multipart/form-data: archivo, tipo, descripcion)
GET	/mis-documentos?tipo=X	Listar mis documentos (filtro opcional por tipo)
GET	/documentos/{id}	Descargar mi documento

Endpoints de documentos en Portal Docente

Prefijo: /v2/portal/docente

Metodo	Ruta	Descripcion
POST	/documentos	Subir documento propio
GET	/mis-documentos?tipo=X	Listar mis documentos
GET	/documentos/{id}	Descargar mi documento

Tipos de documento disponibles: formula_69a , escolaridad , constancia_convenio , cedula , titulo , otro

Procesamiento automatico de imagenes:

- Conversion a WebP (85% calidad)
- Correccion de rotacion EXIF
- Redimensionado a max 2000px
- PDFs se guardan sin modificar

17. Enums y valores posibles

RolUsuario

"estudiante" | "docente" | "administrativo"

EstadoInscripcionMateria

"cursando" | "exonerado" | "a_examen" | "aprobado" | "reprobado" | "perdido_inasistencia" | "abandono"

EstadoInscripcionExamen

"inscripto" | "aprobado" | "reprobado" | "ausente"

TipoPrograma

"carrera" | "curso_corto" | "taller" | "diploma"

AreaPrograma

"administracion" | "comunicacion" | "cultura" | "general" | "informatica"

TipoPreviatura

"aprobada" (requiere APROBADO o EXONERADO) | "exonerada" (requiere solo EXONERADO)

RolDocente

"titular" | "adjunto" | "asistente"

EstadoInstanciaCursado

"planificada" | "en_curso" | "finalizada" | "cancelada"

EstadoInscripcionPrograma

"activa" | "suspendida" | "completada" | "baja"

ModalidadExamen

"presencial" | "virtual" | "hibrido"

TipoExamen

"ordinario" | "extraordinario"

EstadoInstanciaExamen

"programado" | "en_curso" | "finalizado" | "cancelado"

TipoNota

"numerica" | "letra" | "escala_custom"

CargoDocente

"titular" | "interino" | "contrato"

DedicacionDocente

"tiempo_completo" | "medio_tiempo" | "horas"

TipoDocumento

"formula_69a" | "escolaridad" | "constancia_convenio" | "cedula" | "titulo" | "otro"

Errores comunes

Todos los errores siguen este formato:

```
{
  "detail": "Mensaje descriptivo del error"
}
```

Codigo	Significado
400	Validacion fallida (previaturas, periodo, datos invalidos)
401	Token invalido, expirado o revocado
403	Rol insuficiente o recurso no propio
404	Recurso no encontrado
422	Error de validacion de Pydantic (campos faltantes o tipos incorrectos)